

du temps, en reconstituant le calendrier effectif des séances (et non un simple comptage de lignes), en détectant les doubles réservations, et en croisant avec les capacités et surfaces des locaux — restitué dans un dashboard interactif filtrable par ville, site, bâtiment et semestre.

Fichiers sources

Rien d'autre n'est nécessaire pour relancer le pipeline depuis zéro.

FICHIER	CONTENU
Extraction CC au 07.09.csv	Export brut de l'emploi du temps (cours, salles, horaires, colonne PERIODE listant les semaines ISO réelles)
EXP_SALLE.csv	Table CODE → CAPA, capacité de chaque salle
Surfaces (1).xlsx	Surfaces par local — premier essai, incomplet (377/537 salles résolues)
SURFACES GENERAL UPPA 2026_09.xlsx	Référence complète : code bâtiment, ville, site, surfaces par étage

Pipeline de traitement

Quatre scripts PowerShell, à exécuter dans cet ordre. En pratique,

`transform_nom_sal.ps1` puis `build_dedup.ps1` suffisent : ce dernier fait tout le reste en interne.

1 Éclatement et enrichissement

`transform_nom_sal.ps1`

Part de Extraction CC au 07.09.csv, produit Extraction CC au 07.09 - avec salles.csv :

- Éclate NOM_SAL (format Nom (BÂTIMENT-ÉTAGE), plusieurs salles séparées par @) en 3 colonnes salles/batiments/etages
- **Duplique la ligne** une fois par salle quand un cours a plusieurs salles en parallèle — NOM_SAL/UID_SAL/CODE_SAL répartis en parallèle (vérifié alignés sur tout le fichier)
- Convertit DUREE du format HhMM vers un décimal (2h45 → 2.75)
- Ajoute `id_cours` (identifiant de la ligne source, avant duplication) et CAPA (jointure CODE_SAL = CODE de EXP_SALLE.csv)

Produit aussi `Taux_utilisation_par_salle.csv`, enrichi ensuite de `Surface_etage_m2` via une table de correspondance entre nos codes bâtiment abrégés et ceux du fichier Excel (ex. B45→B4B5, DAL_R→DAL, GTR→RT).

2 Reconstitution du calendrier réel

`build_occurrences.ps1`

La colonne PERIODE (ex. [3..6,8..13,16]) liste les vraies semaines ISO où un cours récurrent a lieu : une ligne du CSV n'est **pas** une occurrence unique, c'est un modèle répété. Les heures réelles = DUREE × nombre de semaines dans PERIODE. Exclut les blocages administratifs (durée ≥ 8h, ex. « Réservation de salles »).

3 Détection des chevauchements

`build_conflicts.ps1`

Pour chaque (salle, date), détecte les paires d'occurrences dont les créneaux se recouvrent. Vérifie que CODE_SAL est identique des deux côtés — confirme une vraie double-réservation, pas une collision de nom entre deux salles physiquement différentes.

4 Dédoublonnage et régénération finale

`build_dedup.ps1`

Sélection gloutonne des créneaux non chevauchants par salle/date (garde le maximum de créneaux valides, triés par heure de fin). Recalcule **tout** à partir des seules occurrences conservées : `dashboard_data.json`, `Taux_utilisation_par_salle.csv`, `heatmap_data.json`, `heatmap_par_salle.json` (+ variantes `_s1/_s2`), `weekly_data.json` (+

`weekly_par_salle.json`), `conflicts_data.json` (doit être vide après coup).

Piège PowerShell — `ConvertTo-Json` sérialise un tableau à un seul élément comme un objet simple, pas comme `[objet]`. Toujours envelopper avec `@( ... )` avant d'exporter un tableau qui pourrait n'avoir qu'un élément — sinon le JS plante au premier `.forEach()`.

Le dashboard

`dashboard_template.html` — page unique (CSS+JS inline) qui charge les fichiers JSON ci-dessus via `fetch()`.

01

Vue d'ensemble

KPIs, classement par bâtiment, répartition par tranche de taux, salles à optimiser, tableau détaillé triable

02

Grille horaire

Heatmap par salle et créneau, toujours sur l'année complète

03

Analyse hebdomadaire

Graphique par semaine ISO, heatmap jour×heure, semaines de pointe, jour de pointe

04

Synthèse exécutive

KPIs consolidés, insights texte, méthodologie, classement bâtiments

05

Chevauchements

Doubles-réservations détectées — vide depuis le dédoublonnage

Filtres globaux (communs à tous les onglets) : ville, site, bâtiment — avec groupement **STEE / SSH / EEI / Autres** à cases à cocher de groupe —, recherche libre, taux d'occupation, et sélecteur Année/1er sem./2nd sem. Principe : chaque section calcule `data.filter(matchesGlobal)` puis agrège à partir de là ; aucune donnée pré-agrégée par bâtiment fixe (sauf la Grille horaire, volontairement toujours annuelle).

Chronologie des modifications

Dans l'ordre où elles ont été demandées et livrées.

Fonctionnalité ajoutée

Correctif

Ajustement

CONSTRUCTION INITIALE

- 1 **Structure de base** — 4 onglets, bandeau de filtres ville/site/bâtiment/recherche/taux + sélecteur Année/S1/S2
- 2 **Onglet Chevauchements** — détection des doubles-réservations via `build_conflicts.ps1`, tableau avec case « afficher les groupes fusionnés »

FILTRES ET NAVIGATION

- 3 **Retrait de « Collège »** — mot dans l'en-tête, puis suppression d'un onglet « Collège » (STEE/SSH) ajouté entre-temps par une autre session
- 4 **Filtre bâtiment groupé** — cases à cocher STEE / SSH / EEI / Autres au-dessus des bâtiments individuels ; cocher un groupe sélectionne tous ses bâtiments
- 5 **Répartition des groupes corrigée** — une fusion accidentelle de SSH+EEI par une autre session avait vidé EEI ; répartition restaurée
- 6 **Filtres → Chevauchements** — table bâtiment→ville/site ajoutée pour que ville/site/bâtiment/recherche filtrent aussi cet onglet

LISIBILITÉ

- 7 **Répartition par tranche** — % sous chaque barre + paragraphe explicatif
- 8 **En-têtes horaires explicites** — 8h-9h...17h-18h au lieu de 8h...17h
- 9 **Graphique hebdomadaire** — valeurs affichées (% en haut, semaine en bas), puis échelle absolue 0-100% au lieu d'une échelle relative au maximum filtré
- 10 **Carte « Jour de pointe »** — classement Lundi→Vendredi par taux moyen
- 11 **Semaines de pointe en %** — remplace l'affichage en heures brutes, tri par taux

CORRECTIONS DE FOND

- 12 **Synthèse exécutive ignorait les filtres** — calculait toujours sur les 537 salles ; corrigé pour utiliser `matchesGlobal` comme les autres onglets
- 13 **Bug de plantage complet** — `ConvertTo-Json` sérialisait un objet seul au lieu d'un tableau à 1 élément, plantant le rendu d'Analyse hebdomadaire ET Synthèse exécutive en même temps (même chaîne `renderAll()`) ; corrigé côté script (`@( . . . )`) + filet de sécurité JS
- 14 **Analyse hebdomadaire limitée à « un bâtiment ou tous »** — refonte complète : `weekly_par_salle.json` généré (données par salle individuelle) et agrégation dynamique sur l'ensemble filtré, vérifiée par filtre (bâtiment, groupe STEE, ville, recherche, taux)
- 15 **Heatmap et Jour de pointe ignoraient le semestre** — génération de

heatmap_par_salle_s1.json/_s2.json, branchement selon Année/1er sem./2nd sem.

PUBLICATION

- 16 **Artifact Claude** — publication multi-fichiers (index.html + JSON), republiée à chaque correctif
- 17 **Versions standalone locales** — script d'injection des JSON dans le HTML pour un fichier unique ouvrable sans serveur
- 18 **Dépôt GitLab** — scripts + template propre uniquement, .gitignore excluant toutes les données sensibles

Pièges à éviter en reproduisant

Tableaux PowerShell à 1 élément — ConvertTo-Json les sérialise comme un objet simple. Toujours @(. . .) avant d'exporter.

Cases à cocher de groupe — nécessitent une capture d'événement en amont de l'écouteur générique, sinon « STEE » est traité comme un code bâtiment invalide.

Toute nouvelle section du dashboard doit repartir de `data.filter(matchesGlobal)`, jamais d'un agrégat pré-calculé par bâtiment fixe — sinon elle finira par ignorer un des filtres.

Fichiers finaux

Dashboard en ligne claude.ai/artifact/LbV2u61pjjQXwbNJf42iGX

Version locale Occupation_Salles_UPPA.html

Template propre dashboard_template.html

Dépôt GitLab git.univ-pau.fr/jbmouyabi/analyse_methodologique_2026